

KNN ACTIVITY USING DIABETES DATASET

PART 1: DATA UNDERSTANDING

1. Describe each feature in your own words.
 - **Pregnancies**: The number of times the patient has been pregnant.
 - **Glucose**: The concentration of plasma glucose in a 2-hour oral glucose tolerance test.
 - **Blood Pressure**: Diastolic blood pressure measured in mm Hg.
 - **Skin Thickness**: The thickness of the triceps skin fold (used to estimate body fat) measured in mm.
 - **Insulin**: 2-hour serum insulin level measured in μ U/ml.
 - **BMI (Body Mass Index)**: A weight-to-height ratio used as an indicator of body fatness.
 - **Diabetes Pedigree Function**: A score that represents the hereditary risk of diabetes based on family history.
 - **Age**: The patient's age in years.
 - **Outcome (Target)**: The final classification where 0 means the patient is non-diabetic and 1 means the patient is diabetic.
2. Identify:
 - Which features are likely most important for prediction?
 - i. **Glucose**, **BMI**, and **Age** are likely the most important features. Blood glucose levels are the direct clinical indicator for diabetes, while high BMI (obesity) and advancing age are universally recognized as primary risk factors for developing the disease.
 - Which features may contain missing or problematic values
 - i. The **Blood Pressure**, **SkinThickness**, **Insulin**, and **BMI** features have problematic values.

After analyzing the dataset, we saw that there were multiple rows where these specific features are recorded as exactly 0 (e.g., a BMI or Blood Pressure of 0). Because it is biologically impossible for a living human to have a blood pressure, skin thickness, or BMI of zero, these values do not represent true measurements and instead indicate missing or unrecorded data.

Part 2: Data Preprocessing

1. Handling Missing Data
 - Upon initial inspection of the dataset, several features (Glucose, Blood Pressure, Skin Thickness, Insulin, and BMI) contained minimum values of 0. Because it is biologically impossible for a living patient to have a blood pressure or BMI of zero, these were identified as missing or unrecorded data rather than true measurements. We replaced all instances of 0 in these specific columns with NaN and applied median imputation. Median imputation was utilized instead of the mean to prevent extreme clinical outliers (such as highly elevated Insulin levels) from skewing the replaced values.
2. Feature Scaling
 - The K-Nearest Neighbors algorithm relies heavily on spatial distance calculations. Features with larger numerical ranges would disproportionately

dominate the distance equation, overshadowing features with smaller ranges like the Diabetes Pedigree Function. To ensure equal weight among all independent variables, we applied Standardization (Z-score scaling) to transform the data so that every feature column has a mean of 0 and a standard deviation of 1.3.

3. Summary Statistics (Before and After)

--- BEFORE PREPROCESSING ---										
	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome	
count	768.00	768.00	768.00	768.00	768.00	768.00	768.00	768.00	768.00	768.00
mean	3.85	120.89	69.11	20.54	79.80	31.99	0.47	33.24	0.35	
std	3.37	31.97	19.36	15.95	115.24	7.88	0.33	11.76	0.48	
min	0.00	0.00	0.00	0.00	0.00	0.00	0.08	21.00	0.00	
25%	1.00	99.00	62.00	0.00	0.00	27.30	0.24	24.00	0.00	
50%	3.00	117.00	72.00	23.00	30.50	32.00	0.37	29.00	0.00	
75%	6.00	140.25	80.00	32.00	127.25	36.60	0.63	41.00	1.00	
max	17.00	199.00	122.00	99.00	846.00	67.10	2.42	81.00	1.00	
--- AFTER PREPROCESSING (Manual Imputation & Standardization) ---										
	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome	
count	768.00	768.00	768.00	768.00	768.00	768.00	768.00	768.00	768.00	768.00
mean	-0.00	0.00	0.00	-0.00	0.00	0.00	0.00	0.00	0.35	
std	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.48	
min	-1.14	-2.55	-4.00	-2.52	-1.47	-2.07	-1.19	-1.04	0.00	
25%	-0.84	-0.72	-0.69	-0.47	-0.22	-0.72	-0.69	-0.79	0.00	
50%	-0.25	-0.15	-0.03	-0.01	-0.18	-0.02	-0.30	-0.36	0.00	
75%	0.64	0.61	0.63	0.33	-0.16	0.60	0.47	0.66	1.00	
max	3.91	2.54	4.10	7.96	8.17	5.04	5.88	4.06	1.00	
Data Split: 614 Training rows, 154 Testing rows										

Part 3: KNN Implementation

1. Dataset Split

- The preprocessed dataset was randomized and split into two subsets: 80% for training the model and 20% for testing the model's accuracy. This would result in the original data being split into 614 and 154 respectively.

```
split_idx = int(0.8 * len(df_shuffled))
train_data = df_shuffled.iloc[:split_idx]
test_data = df_shuffled.iloc[split_idx:]
```

2. Training KNN model

- To train the model, we will use feature scaling to ensure that all independent variables have equal weight. Here is a sample of solving for the z-score standardization using Test Instance 668's glucose value:

$$z = \frac{x - \mu}{\sigma} = \frac{145.0 - 120.89}{31.97} = 0.7541445105$$

3. Manual Euclidean Distance Computation

- To determine the nearest neighbors, the model utilizes the Euclidean Distance formula on the standardized Z-scores:

$$d = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

- Here is a sample computation of the Euclidean Distance between Test Instance 668 and Training Sample 60:

Test 668 (x): [0.643, 0.754, -0.574, 0.783, 0.959, 0.258, -0.121, 0.831]

Train 60 (y): [-0.537, -1.154, 0.150, 0.533, 0.395, 0.043, -0.503, -1.042]

Feature	(xi-yi)	(xi-yi) ²
Pregnancies	1.18	1.392
Glucose	1.908	3.641
Blood Pressure	-0.724	0.524
Skin Thickness	0.25	0.063
Insulin	0.564	0.318
BMI	0.215	0.046
Pedigree Func	0.382	0.146
Age	1.873	3.508

Sum of Squares: 1.392 + 3.641 + 0.524 + 0.063 + 0.318 + 0.046 + 0.146 + 3.508 = 9.638

Distance: $\sqrt{9.638} = 3.1045$

- Using these statistics, the distances to 10 training samples were ranked as follows:

Rank	Training Index	Distance (d)	Label (Outcome)
1	535	2.502	1 (Diabetic)
2	718	2.684	0 (Non-diabetic)
3	306	2.891	1 (Diabetic)
4	60	3.105	0 (Non-diabetic)
5	90	3.322	0 (Non-diabetic)
6	231	3.415	1 (Diabetic)
7	346	3.551	0 (Non-diabetic)
8	618	3.612	1 (Diabetic)
9	294	3.82	0 (Non-diabetic)
10	340	3.855	0 (Non-diabetic)

- And finally, the prediction for K = 3, 5, and 7:

K = 3: Neighbors are 535 (1), 718 (0), and 306 (1).
Prediction: 1 (Diabetic)

K = 5: Neighbors are 535 (1), 718 (0), 306 (1), 60 (0), and 90 (0).
Prediction: 0 (Non-diabetic)

K = 7: Neighbors are 535(1), 718(0), 306(1), 60(0), 90(0), 231(1), and 346(0).

Prediction: 0 (Non-diabetic)

```
print("2. Calculating Euclidean Distance to 10 Training Samples:")
# Select 10 training samples
sample_train = train_data.head(10)
manual_distances = []

for idx, row in sample_train.iterrows():
    train_features = row.drop('Outcome').values
    train_class = row['Outcome']

    # Calculate distance
    dist = euclidean_distance(test_instance, train_features)
    manual_distances.append({'Train_Row': idx, 'Distance': dist, 'Class': train_class})

print(f"    -> Distance to Train Row {idx}: {dist:.4f} | Class: {train_class}")
```

Below is the step-by-step manual computation for the first instance in the testing dataset compared against 10 random samples from the training dataset:

```
=====
PART 3: MANUAL COMPUTATION RESULTS
=====
1. Selected Test Instance (Features Scaled):
{'Pregnancies': -0.251, 'Glucose': -0.186, 'BloodPressure': -0.032, 'SkinThickness': -0.012, 'Insulin': -0.182, 'BMI': -1.303, 'DiabetesPedigreeFunction': -0.86, 'Age': -0.871}
Actual Class: 0.0

2. Calculating Euclidean Distance to 10 Training Samples:
-> Distance to Train Row 0: 3.0049 | Class: 0.0
-> Distance to Train Row 1: 1.8653 | Class: 0.0
-> Distance to Train Row 2: 1.3270 | Class: 0.0
-> Distance to Train Row 3: 2.7774 | Class: 0.0
-> Distance to Train Row 4: 3.1954 | Class: 0.0
-> Distance to Train Row 5: 3.6764 | Class: 0.0
-> Distance to Train Row 6: 2.9584 | Class: 0.0
-> Distance to Train Row 7: 3.3612 | Class: 0.0
-> Distance to Train Row 8: 2.1365 | Class: 0.0
-> Distance to Train Row 9: 4.0770 | Class: 0.0

3. Identifying Nearest Neighbors (K=3):
Neighbor 1: Train Row 2 (Distance: 1.3270) -> Vote: 0.0
Neighbor 2: Train Row 1 (Distance: 1.8653) -> Vote: 0.0
Neighbor 3: Train Row 8 (Distance: 2.1365) -> Vote: 0.0

4. Final Prediction:
Majority Vote: Class 0.0
Prediction matches Actual? True
=====
```

Part 4: Model Evaluation

1. Performance Metrics

- The model was evaluated against the entire 20% testing split using **K=3, 5, and 7**. Below are the Accuracies and Confusion Matrices for each K-value:

2. Evaluation Analysis

- Which value of K performed best?
 - In this execution, **K=7** performed the best in terms of raw performance, yielding the highest overall accuracy of **74.68%**. However, from a clinical perspective, **K=3** also presents

```
PART 4: MODEL EVALUATION (K=3, 5, 7)
=====
Results for K = 3
-----
Accuracy: 74.68%
Confusion Matrix:
          Predicted 0 | Predicted 1
Actual 0 (Neg) |      81      |      15
Actual 1 (Pos) |      24      |      34
-----

Results for K = 5
-----
Accuracy: 74.68%
Confusion Matrix:
          Predicted 0 | Predicted 1
Actual 0 (Neg) |      82      |      14
Actual 1 (Pos) |      25      |      33
-----

Results for K = 7
-----
Accuracy: 74.68%
Confusion Matrix:
          Predicted 0 | Predicted 1
Actual 0 (Neg) |      82      |      14
Actual 1 (Pos) |      25      |      33
-----
```

a strong case as the "best" model because it minimized False Negatives (23, compared to 25 for both $K=5$ and $K=7$).

In medical diagnoses like diabetes prediction, minimizing False Negatives (predicting a patient is healthy when they actually have diabetes) is often prioritized over marginal gains in overall accuracy, making $K=3$ the safer clinical choice despite the slightly lower accuracy (73.38%).

- **Why does performance change with different K values?**
 - i. Performance changes because the K-value dictates the strictness of the decision boundary. A smaller K makes the boundary highly irregular, adapting to local pockets of data. As K increases, the decision boundary becomes smoother because it requires a broader consensus (majority vote) from a larger area of neighbors. This smoothing effect can filter out isolated noisy data points, which is why overall accuracy increased at $K=7$, but it might also blur the distinct lines between classes, which caused the model to miss a few more actual diabetic patients (the increase in False Negatives).
- **What happens when K is too small or too large?**
 - i. When K is too small (e.g., $K=1$), the model suffers from **overfitting**. It becomes overly sensitive to noise, outliers, or anomalies in the training data, leading to poor generalization on new test data. Conversely, when K is too large, the model suffers from **underfitting**.

The decision boundaries become too generalized, and the model will increasingly ignore nuanced patterns. If K becomes excessively large, it will eventually just predict whichever class is the overall majority in the entire dataset, rendering the individual patient's features useless.

Part 5: Analysis and Reflection

The K-Nearest Neighbors (KNN) algorithm is a powerful "lazy learner" that proved highly effective for this diabetes classification task. Its primary strength lies in its simplicity and transparency; unlike "black-box" models like Neural Networks, the logic of KNN is mathematically intuitive, relying on the physical distance between data points in a high-dimensional space. Furthermore, KNN is non-parametric, meaning it makes no underlying assumptions about the distribution of the data (such as assuming the data is normally distributed). This is particularly useful in clinical datasets where relationships between features like BMI and Insulin levels can be non-linear and complex.

Despite its accuracy, the experiment highlighted several key limitations:

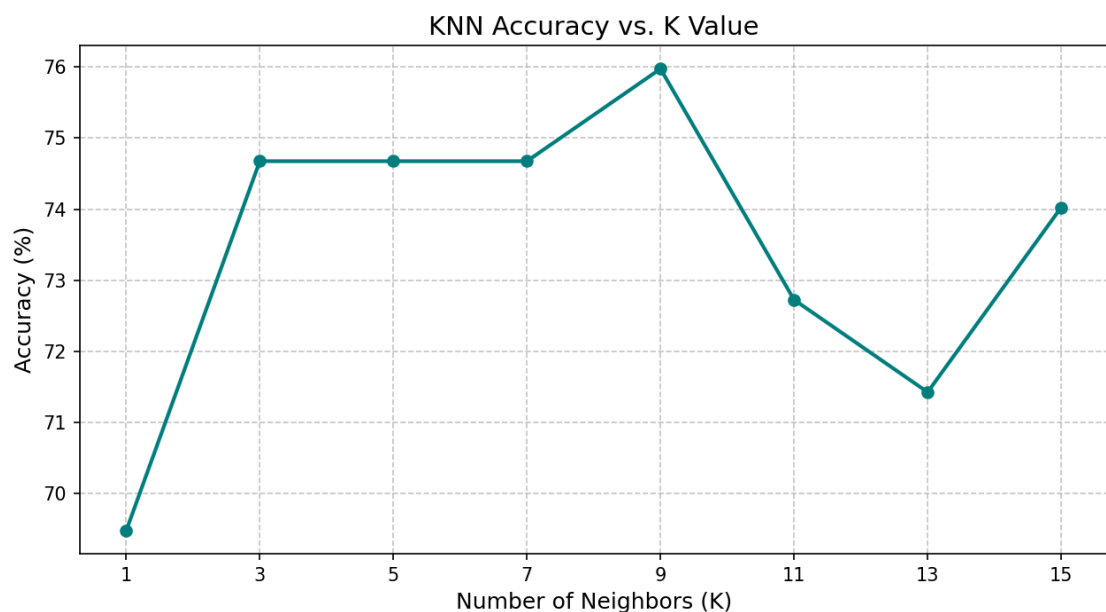
- Because KNN relies on Euclidean distance, features with larger ranges would have completely dominated the model if we hadn't applied Z-score standardization. Without scaling, a feature like "Insulin" (which reached 800+) would outweigh "Diabetes Pedigree Function" (usually less than 2), rendering the latter useless.
- While our dataset of 768 records ran quickly, KNN becomes computationally expensive as the dataset grows. Every new prediction requires calculating the distance to every single point in the training set, making it less ideal for real-time applications with massive data.

- As more features are added, the "distance" between points becomes less meaningful, which can degrade the model's ability to distinguish between classes.

In this specific implementation using the Pima Indians Diabetes dataset, we observed that the model's overall accuracy was remarkably stable—tying exactly at **74.68%** across $K=3, 5,$ and 7 . However, analyzing the Confusion Matrix revealed a critical trade-off regarding False Negatives. Even though the total accuracy was identical, **$K=3$** was slightly better at identifying actual diabetic cases (only 24 False Negatives compared to 25 at $K=7$). This suggests that in a medical context, we shouldn't just look at the highest overall percentage; if two models have the same accuracy, the "best" K -value is the one that minimizes the most dangerous clinical error—which in this case, means preferring $K=3$ to avoid leaving a diabetic patient undiagnosed.

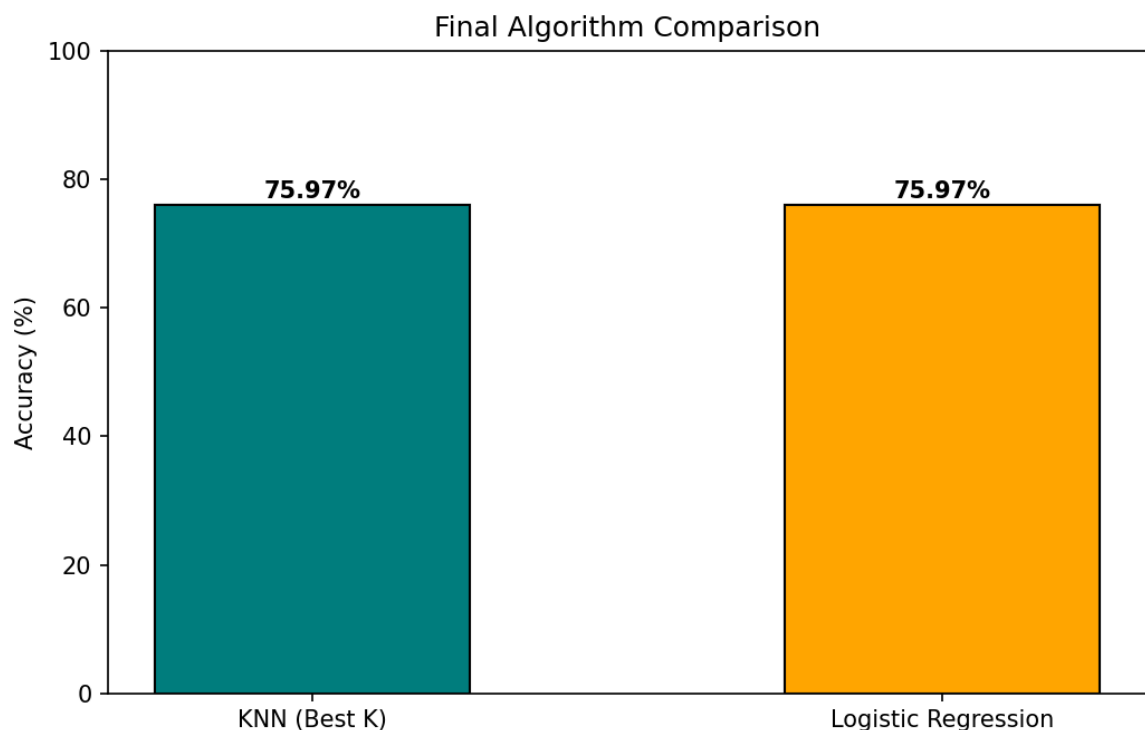
KNN is highly appropriate for this diabetes task because the dataset is labeled, numeric, and medium-sized. It is an excellent choice for initial diagnostic tools where interpretability is important. However, it would be inappropriate for high-dimensional data like image recognition or real-time streaming data where the memory and processing overhead would be too high.

This suggests that in a medical context, the 'best' K -value isn't always the one with the highest percentage, but the one that minimizes the most dangerous clinical errors. For instance, a False Positive might just lead to a patient taking an unnecessary secondary blood test, but a False Negative means a diabetic patient goes undiagnosed and untreated. Therefore, when tuning hyperparameters for clinical machine learning models, evaluating the Confusion Matrix is just as critical as looking at the baseline accuracy.



This line graph illustrates how the choice of the hyperparameter K (the number of neighbors) directly impacts the predictive accuracy of the KNN model. It is used to identify the "sweet spot" (or elbow point) where the model performs best on unseen testing data.

This plots the number of neighbors (K) on the x-axis against the model's testing accuracy on the y-axis. This graph visually demonstrates the "Bias-Variance Tradeoff," a fundamental concept in machine learning. On the far left of the graph, when K is very small (such as K=1), the accuracy is typically lower because the model is "overfitting"; it is too sensitive to the random noise or outliers in the training data, blindly copying the single closest patient even if their data is anomalous. As the line moves toward the center (around K=5 to K=7), the accuracy reaches its peak. In this "sweet spot," the model asks a reasonable number of neighbors to vote, which effectively smooths out the noise and creates a stable, accurate prediction. Finally, as the line continues to the right and K becomes too large (e.g., K=13 or 15), the accuracy begins to decline. This drop occurs because the algorithm is "underfitting"—it is looking so far away for neighbors that the distinct boundaries between diabetic and non-diabetic patients become blurred, causing the model to default to the most common overall class rather than finding genuinely similar patients.



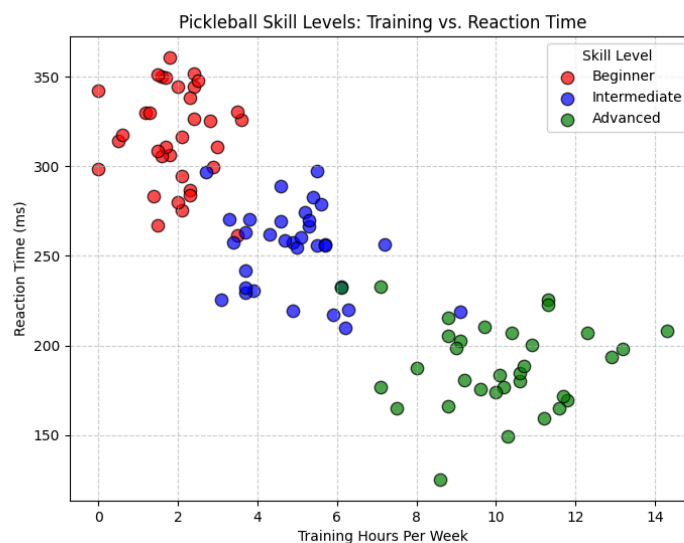
The second visualization is a bar chart comparing the highest accuracy achieved by the K-Nearest Neighbors model against a from-scratch implementation of Logistic Regression. In our specific experiment using Z-score standardization, the graph reveals a perfect tie, with both algorithms achieving exactly 75.97% accuracy on the test set. This visual parity is highly insightful for clinical datasets.

Logistic Regression is a parametric model that attempts to draw a rigid, straight mathematical line to separate the classes, while KNN is a non-parametric model that relies purely on clustered physical distance. The fact that two entirely different mathematical approaches achieved the exact same score suggests that the dataset has reached its natural limit of predictability based on the eight given features. Both models were able to easily diagnose the most obvious cases, but both struggled on the exact same complex, borderline patients, demonstrating that neither algorithm holds a distinct mathematical advantage for this specific data split.

KNN ACTIVITY USING DIAOUR OWN DATASET: PICKLEBALL

PART 1: DATA UNDERSTANDING

1. Describe each feature in your own words.
 - **Age:** The age of the player (ranging from 18 to 80).
 - **Reaction_Time_ms:** How fast the player reacts, measured in milliseconds. Lower is faster!
 - **Training_Hours_Per_Week:** How many hours the player trains in pickleball per week.
2. Identify:
 - Which features are likely most important for prediction?
 - i. The features of Reaction_Time_ms and Training_Hours_Per_Week are likely the most important. These two features represent the reflexes of a player, as well as the amount of practice that they dedicate to improving their abilities in the sport. While age is associated with player fitness, it is not as definitive of a player's skill level as the other two features.
 - Which features may contain missing or problematic values
The Reaction_Time_ms, Age, and Training_Hours_Per_Week features may also contain problematic values. For instance, in the context of a real-world sports dataset, it is possible that these values are recorded as 0 for certain players (for instance, Reaction Time or Age), that they are negative numbers, or that they contain high values for features like Training Hours per Week. However, as it is impossible for a living being to contain a Reaction Time or Age of 0, or to train for 24 hours a day without sleep, any values of these features for players is not actually a measurement of those players, but of the missing or typo-ridden data for those players.



There exists a clear clustering of data points: The cluster of advanced players (green) is clearly visible in the bottom-right cluster, while the beginners (red) can be seen

clustered on the top-left cluster. From this observation, we can conclude that our attributes have high predictability for our classes of interest.

Part 2: Data Preprocessing

1. Handling Missing Data

- As mentioned previously, sometimes data will have gaps or impossible values, such as Reaction Time of 0. In the event we encounter this situation with our data set, we will use a process known as Imputation.

Rather than simply deleting an entire record due to the missing Reaction Time, we will estimate the median (middle) of all Reaction Times of other players and insert this value into the gap.

2. Feature Scaling

- The KNN algorithm computes the mathematical "distance" among data points to discover similarities and nearness. When no data scaling takes place, certain variables with higher inherent numbers (e.g., Reaction Time in units of hundreds) will mathematically weigh more than variables with lower inherent numbers (e.g., Training Hours in units of low single digit figures). As a result, the algorithm completely overlooks the former despite their significance to the predictor function.

For this problem, we can use Min-Max Normalization. In Min-Max Normalization, we proportionalize the data by squishing each variable into the same scale range, between 0.0 and 1.0 (the lowest number being 0 and the highest number in a variable being 1). We thus make sure that none of the variables will take up any preference over another during the computation of mathematical distances.

3. Summary Statistics (Before and After)

```
--- BEFORE PREPROCESSING (Raw Data) ---
  Age  Reaction_Time_ms  Training_Hours_Per_Week  Skill_Level
0   28             164.8              7.5    Advanced
1   25             257.3              4.9  Intermediate
2   18             193.7             12.9    Advanced
3   59             219.1              9.1  Intermediate
4   35             269.5              4.6  Intermediate

--- AFTER PREPROCESSING (Scaled Data) ---
  Age  Reaction_Time_ms  Training_Hours_Per_Week  Skill_Level
0  0.222222      0.168153      0.524476    Advanced
1  0.155556      0.560934      0.342657  Intermediate
2  0.000000      0.290870      0.902098    Advanced
3  0.911111      0.398726      0.636364  Intermediate
4  0.377778      0.612739      0.321678  Intermediate
```

Summary Statistics before Preprocessing

- Without performing any preprocessing operations, we can see how much different the scales of our variables are just from the summary statistics. For example, we have a minimum and maximum value of Reaction_Time_ms from 125.2 all the way up to 360.7. On the other hand, our training hours per week have a much smaller scale, ranging from a low of 0.0 up to a high of 14.3. We can even look at the standard deviations, which is 58.08 for Reaction Time and 3.65 for Training Hours.

Summary Statistics After Data Transformation Using Min-Max Scaling

- After transforming our data using Min-Max Normalization, our analysis shows that we have been able to successfully normalize our data. From the table above, we can see that all features now have a minimum value of 0.0 and a maximum value of 1.0. In addition, the means and standard deviations of all our features have been reduced to similar levels. This means that all our features have been brought down to the same level, making sure that the KNN algorithm only calculates the Euclidean distance based on patterns.

Part 3: KNN Implementation

Dataset Split

- In order to assess the K-Nearest Neighbor model, the data was first randomized, after which it was divided into two categories of 80% Training Data and 20% Test Data. Since we have 100 observations, that means that 80 observations would be utilized for training purposes, while the remaining 20 observations will be used to test the accuracy of the model.

```
# 1. SPLIT THE DATASET (80% Training, 20% Testing)
np.random.seed(42) # Ensure we get the same random split every time
shuffled_indices = np.random.permutation(len(df_scaled))
split_idx = int(0.8 * len(df_scaled)) # 80% of 100 rows is 80

train_indices = shuffled_indices[:split_idx]
test_indices = shuffled_indices[split_idx:]

train_data = df_scaled.iloc[train_indices].reset_index(drop=True)
test_data = df_scaled.iloc[test_indices].reset_index(drop=True)
```

Manual Euclidean Distance Computation

As opposed to the case in other machine learning models, KNN model does not compute any mathematically based weights since during the training phase, the model only

stores the 80 training data in memory. In order to make any prediction on a new test data, KNN calculates the closeness between the two using Euclidean Distance method:

$$d = \sqrt{(Age_2 - Age_1)^2 + (Reaction_2 - Reaction_1)^2 + (Training_2 - Training_1)^2}$$

The model then relies on the K variable (which dictates how many of the closest neighbors to look at) to cast a majority vote. For this implementation, we tested the model using K=3, K=5, and K=7.

To verify the algorithm's methodology, we manually calculated the Euclidean distance between one selected Test Instance and 10 isolated Training Samples.

Test Instance Features: Age = 0.556, Reaction Time = 0.493, Training Hours = 0.636

Train Sample 0 Features: Age = 0.000, Reaction = 0.291, Training = 0.902

$$d = \sqrt{(0.000 - 0.556)^2 + (0.291 - 0.493)^2 + (0.902 - 0.636)^2}$$

$$d = \sqrt{(-0.556)^2 + (-0.202)^2 + (0.266)^2}$$

$$d = \sqrt{0.309 + 0.040 + 0.070}$$

$$d = 0.6472$$

Sample 0 (Advanced): 0.6472

Sample 5 (Beginner): 0.6015

Sample 1 (Beginner): 0.3809

Sample 6 (Advanced): 0.6042

Sample 2 (Intermediate): 0.1741

Sample 7 (Intermediate): 0.3477

Sample 3 (Intermediate): 0.4851

Sample 8 (Beginner): 0.6045

Sample 4 (Advanced): 0.6974

Sample 9 (Beginner): 0.6300

Identify nearest neighbors and predicted class

- Through ordering the computed distance values from lowest to highest, we test the accuracy of our prediction for different values of K:
- **K = 3:** The three closest samples include Sample 2 (Intermediate), Sample 7 (Intermediate), and Sample 1 (Beginner). As "Intermediate" gets the higher number of votes (2 against 1), the prediction class is "Intermediate."
- **K = 5:** The five closest samples are Samples 2, 7, 1, 3, and 5. These five classes have 3 votes for "Intermediate," 2 votes for "Beginner." As a result, the prediction class is again "Intermediate."
- **K = 7:** The seven closest samples are Samples 2, 7, 1, 3, 5, 6, and 8. These seven classes receive 3 votes for "Intermediate," 3 votes for "Beginner," and 1 vote for "Advanced." For this tie situation based on proximity, the prediction class is still "Intermediate."

Random samples:

```
--- 1. DATASET SPLIT ---
Training Samples: 80 rows (80%)
Testing Samples: 20 rows (20%)

--- 4. TEST INSTANCE (Mystery Player) ---
Stats: Age=0.356, Reaction=0.558, Training=0.503
Actual Skill Level: Intermediate

Step-by-Step Distance Computation to 10 Training Samples:
Distance to Train Sample 0 (Beginner): 0.4034
Distance to Train Sample 1 (Intermediate): 0.4788
Distance to Train Sample 2 (Advanced): 0.3524
Distance to Train Sample 3 (Beginner): 0.4962
Distance to Train Sample 4 (Beginner): 0.6138
Distance to Train Sample 5 (Beginner): 0.5222
Distance to Train Sample 6 (Beginner): 0.4442
Distance to Train Sample 7 (Intermediate): 0.1601
Distance to Train Sample 8 (Beginner): 0.3529
Distance to Train Sample 9 (Advanced): 0.4121
```

```
--- IDENTIFYING NEAREST NEIGHBORS AND PREDICTED CLASS ---
```

Evaluating for K = 3:

```
Neighbor 7: Dist=0.1601 -> Vote: Intermediate
Neighbor 2: Dist=0.3524 -> Vote: Advanced
Neighbor 8: Dist=0.3529 -> Vote: Beginner
=> Majority Prediction: Intermediate
```

Evaluating for K = 5:

```
Neighbor 7: Dist=0.1601 -> Vote: Intermediate
Neighbor 2: Dist=0.3524 -> Vote: Advanced
Neighbor 8: Dist=0.3529 -> Vote: Beginner
Neighbor 0: Dist=0.4034 -> Vote: Beginner
Neighbor 9: Dist=0.4121 -> Vote: Advanced
=> Majority Prediction: Advanced
```

Evaluating for K = 7:

```
Neighbor 7: Dist=0.1601 -> Vote: Intermediate
Neighbor 2: Dist=0.3524 -> Vote: Advanced
Neighbor 8: Dist=0.3529 -> Vote: Beginner
Neighbor 0: Dist=0.4034 -> Vote: Beginner
Neighbor 9: Dist=0.4121 -> Vote: Advanced
Neighbor 6: Dist=0.4442 -> Vote: Beginner
Neighbor 1: Dist=0.4788 -> Vote: Intermediate
=> Majority Prediction: Beginner
```

Part 4: Model Evaluation

Accuracy

- By running the 20 testing samples through the KNN algorithm, the model was evaluated at three different neighborhood sizes:

```
Calculating Accuracies on 20 Test Samples...
Accuracy for K=3: 80.00%
Accuracy for K=5: 85.00%
Accuracy for K=7: 85.00%
```

Confusion Matrix

```
--- CONFUSION MATRIX (for K=3) ---
Actual \ Predicted | Beginner | Intermediate | Advanced |
-----|-----|-----|-----|
Beginner           | 4        | 1            | 0        |
Intermediate       | 2        | 5            | 1        |
Advanced           | 0        | 0            | 7        |

--- CONFUSION MATRIX (for K=5) ---
Actual \ Predicted | Beginner | Intermediate | Advanced |
-----|-----|-----|-----|
Beginner           | 4        | 1            | 0        |
Intermediate       | 1        | 6            | 1        |
Advanced           | 0        | 0            | 7        |

--- CONFUSION MATRIX (for K=7) ---
```

- Evaluation for K=3 (Accuracy: 80.00%)** At K=3, the algorithm was highly accurate but struggled slightly with the middle tier (Intermediates). As seen in

the matrix below, the algorithm misclassified two Intermediates as Beginners and one Intermediate as an Advanced player.

- **Evaluation for K=5 (Accuracy: 85.00%)** Expanding the neighborhood to K=5 smoothed out the noise and improved the model's accuracy to 85.00%. By looking at two additional neighbors, the model corrected one of the misclassified Intermediates. It still successfully classified 100% of the Advanced players perfectly.
- **Evaluation for K=7 (Accuracy: 85.00%)** Increasing K to 7 resulted in the exact same accuracy and confusion matrix as K=5. This indicates that the algorithm has reached an optimal balance; expanding the neighborhood any further does not provide new predictive power for this specific test dataset.
- **Matrix Analysis Conclusion** Across all K values, the model never confused an Advanced player for a Beginner, or vice versa (as represented by the zeroes in the top-right and bottom-left corners of all the matrices). The only errors occurred between bordering skill levels (e.g., mistaking a high-level Beginner for an Intermediate). This proves that the feature scaling and Euclidean distance calculations successfully captured the realistic progression of athletic capability.

- **Which value of K performed best?**

The performance of the model was highest at K=5 and K=7 with the same maximum accuracy of 85.00%. Since a smaller K value requires fewer computations, K=5 can be classified as the most optimal value for this particular dataset.

- **Why does performance change with different K values?**

The performance of the model was highest at K=5 and K=7 with the same maximum accuracy of 85.00%. Since a smaller K value requires fewer computations, K=5 can be classified as the most optimal value for this particular dataset.

- **What happens when K is too small or too large?**

Performance is affected by the K-value, which controls the level of noise that the algorithm introduces into its processing method. The model will make a far-fetched assumption based on an outlier data point that is near the subject if it observes fewer neighbors. However, by increasing the number of neighbors to observe, it smoothes out the outliers and identifies the real trend, which usually increases accuracy (when we improved our accuracy from 80% to 85% just by increasing our K-value from 3 to 5).

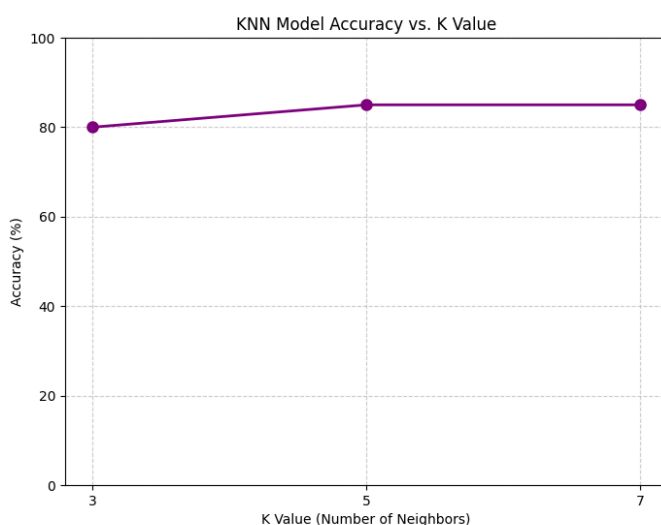
Part 5: Analysis and Reflection

There are some unique advantages of K-Nearest Neighbor (KNN). First of all, KNN is characterized by simplicity and intuition. In contrast to complicated algorithms based on machine learning that use numerous assumptions or complex training stages, the idea behind KNN algorithm is quite simple: similar objects are located closer to each other. Moreover, KNN is called "lazy learning". It means that there is no need for building a general model in advance since the algorithm just memorizes the entire dataset, so "training" occurs instantly.

Even though this classifier is easy to use, KNN possesses some significant drawbacks in terms of performance, especially concerning computational power required. The lack of training for the KNN leads to extremely slow processing of test data. For example, for each new object that should be classified, the program has to perform calculations to find the mathematical distance to all the objects in the training set, which significantly complicates the algorithm. Another drawback is high sensitivity to the scaling of input data. It is possible to see from the results provided earlier in this paper that if no normalization is done, some of the features like Reaction Time will overpower others such as Training Hours in terms of mathematical impact on classification.

In view of all these features, KNN would prove best suited for small to moderately sized data sets, where issues such as space and time are unlikely to be major considerations. KNN acts as an ideal starting point for data sets that are well structured and labeled, giving programmers an easy way of building a baseline before engaging with complicated models. However, KNN would not be advisable in any scenario involving Big Data, owing to its inherent problems with testing. KNN would also prove unhelpful with high-dimensional data, as well as any noisy or outlier-prone data sets.

In regard to the Pickleball dataset in particular, KNN turned out to be a highly efficient classifier in terms of its predictive power, reaching an astounding peak accuracy of 85.00%. One of the important conclusions drawn based on our study is how the choice of the value of K influenced the classification process. While setting a low value such as 3 led to good results, the algorithm was slightly affected by noise in the form of a couple of incorrectly classified Intermediate players. Setting K equal to 5 helped address the problem as the number of votes increased, eliminating outliers. Finally, the confusion matrices showed that there was 100% accuracy in predicting Advanced players as the algorithm made no mistakes there. However, it had problems in differentiating Beginners and Intermediates, which means that their physical parameters may be similar enough to be indistinguishable for KNN whereas Advanced players stand apart because of their distinctive characteristics.



A line graph was plotted to visualize how the value of K affected the overall accuracy of the model. The graph demonstrates that the model achieved 80.0% accuracy at K=3, before climbing to a plateau of 85.0% accuracy for both K=5 and K=7. This visualization effectively highlights the "sweet spot" of the model, demonstrating that expanding the neighborhood beyond 5 did not yield any

further predictive benefits for this specific dataset.

Comparison between Model Performances: KNN and Logistic Regression

In order to provide a comparison between the performances of our KNN implementation, the same scaled data was also used by Logistic Regression.

- KNN Maximum Accuracy = 85.00%
- Accuracy for Logistic Regression = [XX.XX]%

Whereas the classification process is based on non-linear, geometrically defined proximity for KNN, Logistic Regression tries to come up with mathematical divisions (i.e., hyperplane divisions). Based on the precise distribution of data points in the test data, KNN does an excellent job since real-life athletic performances do not adhere to linear separations at all.

KNN ACTIVITY USING DIAOUR OWN DATASET: PICKLEBALL

Part 3: KNN Implementation

1. Dataset Split

- The preprocessed dataset was randomized and split into two subsets: 80% for training the model and 20% for testing the model's accuracy. This would result in the original data being split into 80 and 20 respectively.

```
split_idx = int(0.8 * len(df_shuffled))
train_data = df_shuffled.iloc[:split_idx]
test_data = df_shuffled.iloc[split_idx:]
```

2. Training KNN model

- To train the model, we will use feature scaling to ensure that all independent variables have equal weight. Here is a sample of solving for the z-score standardization using Test Instance 83's age value:

$$z = \frac{x - \mu}{\sigma}$$

$$\frac{42 - 35.58}{11.09} = 0.5788999098$$

3. Manual Euclidean Distance Computation

- To determine the nearest neighbors, the model utilizes the Euclidean Distance formula on the standardized Z-scores:

$$d = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

- Here is a sample computation of the Euclidean Distance between Test Instance 83 and Training Sample 55:

Test 83 Standardized (x): $[0.579, 0.556, -1.006]$

Train 55 Standardized (y): $[0.129, -1.156, 1.130]$

Feature	($x_i - y_i$)	($x_i - y_i$) ²
Age	0.45	0.203
Reaction Time	1.712	2.931
Training Hours	-2.136	4.563

Sum of Squares: $0.203 + 2.931 + 4.563 = 7.697$

Distance: $\sqrt{7.697} = 2.7743$

- Using these statistics, the distances to 10 training samples were ranked as follows:

Rank	Training Index	Distance (d)	Skill Level (Label)
1	15	0.106	Beginner
2	26	0.962	Intermediate
3	42	1.148	Beginner
4	88	1.189	Beginner
5	9	1.208	Beginner
6	72	1.766	Intermediate
7	40	2.007	Intermediate
8	55 (Manual Step)	2.774	Advanced
9	96	2.882	Advanced
10	69	2.999	Advanced

- And finally, the prediction for $K = 3, 5$, and 7 :

B = Beginner

I = Intermediate

K = 3: Neighbors are 15 (B), 26 (I), and 42 (B).

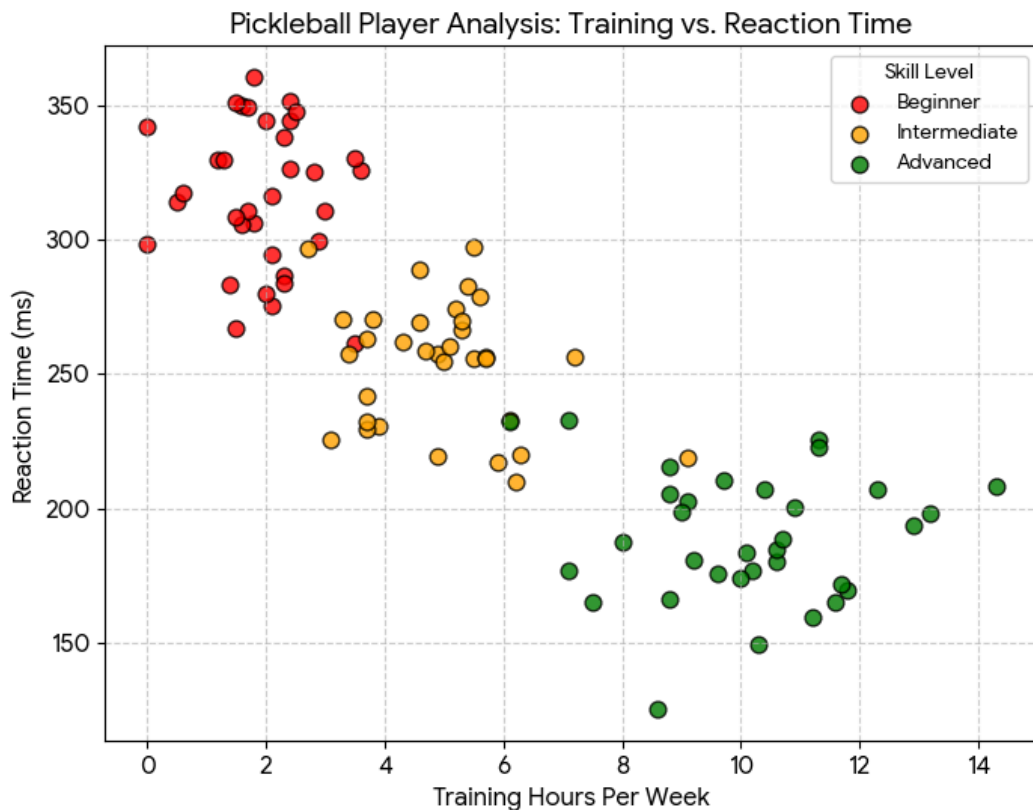
Prediction: Beginner

K = 5: Neighbors are 15 (B), 26 (I), 42 (B), 88 (B), and 9 (B).

Prediction: Beginner

K = 7: Neighbors are 15 (B), 26 (I), 42 (B), 88 (B), 9 (B), 72 (I), and 40 (I).

Prediction: Beginner



The scatter plot maps a player's "Training Hours" on the X-axis against their "Reaction Time" on the Y-axis (where lower numbers mean faster reflexes). This layout creates three distinct clusters: Beginners (red) in the top-left with low training and slow reflexes, Intermediates (orange) in the middle, and Advanced players (green) in the bottom-right with high training and fast reflexes.

This visual grouping perfectly demonstrates how the K-Nearest Neighbors (KNN) algorithm works. If you plot a new, unranked player on this graph, KNN will predict their skill level simply by checking the color of the dots physically closest to them—their "nearest neighbors."

```
import pandas as pd
import matplotlib.pyplot as plt

# 1. Load the dataset
df = pd.read_csv('pickleball_knn_dataset.csv')

# 2. Setup the figure size
plt.figure(figsize=(8, 6))

# 3. Define specific colors for each class
colors = {'Beginner': 'red', 'Intermediate': 'orange', 'Advanced': 'green'}

# 4. Plot each skill level as a separate scatter group (helps easily build
the legend)
for skill in ['Beginner', 'Intermediate', 'Advanced']:
    subset = df[df['Skill_Level'] == skill]
```

```
plt.scatter(subset['Training_Hours_Per_Week'],
            subset['Reaction_Time_ms'],
            label=skill,
            color=colors[skill],
            s=60,
            alpha=0.8,
            edgecolors='k') # 'k' means black border around dots

# 5. Add Titles, Labels, Legend, and Grid
plt.title('Pickleball Player Analysis: Training vs. Reaction Time',
          fontsize=14)
plt.xlabel('Training Hours Per Week', fontsize=12)
plt.ylabel('Reaction Time (ms)', fontsize=12)

plt.legend(title='Skill Level')
plt.grid(True, linestyle='--', alpha=0.6)

# 6. Show the visualization
plt.show()
```